

# SPAR-PIBT, from the inside

Every rule the planner applies, why it is there, and the arithmetic it does each timestep.

**The planner in one paragraph.** Robots live on a 4-connected grid of a warehouse. Every timestep each robot picks one of its five moves — four neighbours or stay — and PIBT resolves the fights: higher-priority robots claim cells first, and a robot that is in the way is *pushed* out of the way rather than merely blocked. SPAR adds a layer above that: corridors ("aisles") are given a traffic direction that reverses on demand, and a robot that drives against the current is **not forbidden** — **it is charged**. The charge is a number in the same objective that ranks every other move.

**The one design decision.** Direction is a term in the objective, not a constraint on the domain. PIBT's guarantee rests on a robot always having somewhere to be pushed; a one-way rule deletes exactly those escape moves. Pricing keeps them, and the price is set so a robot pays it only when nothing better is left.

## 1 The pieces, and where they live

Five objects, and every formula below refers only to these.

| OBJECT       | WHAT IT IS                                                                                                                                                                                                                                                  | CODE                      |
|--------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------|
| $G = (V, E)$ | the passable grid, 4-connected. Vertices are cells.                                                                                                                                                                                                         | <code>graph.py</code>     |
| aisle $a$    | a maximal <i>straight</i> run of non-intersection cells, ordered from <code>start_vertex</code> to <code>end_vertex</code> . Moving up the index is <b>FORWARD</b> , down is <b>REVERSE</b> . Bends are split, so an aisle always lies on one compass axis. | <code>warehouse.py</code> |
| robot $i$    | position $x_i$ , a task, a waypoint $g_i$ , a route, and a priority $p_i$ .                                                                                                                                                                                 | <code>robot.py</code>     |
| task $\tau$  | a pickup cell and a delivery cell, released at some timestep. Lifelong: they keep arriving (Poisson).                                                                                                                                                       | <code>task.py</code>      |
| timestep     | all robots move simultaneously; a move is to a neighbour or stay.                                                                                                                                                                                           | <code>simulator.py</code> |

Two derived facts about the map matter later. A cell is a **bottleneck** if it is a cut vertex, an endpoint of a bridge edge, or a dead end. An aisle is **manageable** — allowed to hold a traffic direction — only if it is at least `directional_aisle_min_length` = 4 cells long and contains no bridge edge. Making a cut edge one-way would disconnect the warehouse, so those aisles stay permanently open. That check is the reason the system cannot lock itself out of a region.

## 2 One timestep, end to end

This is `Simulator.step()`, in order. Steps 1–3 are the lifelong bookkeeping; 4–8 are SPAR; 11–12 are PIBT; 13–14 execute.

| #     | WHAT HAPPENS                                                                               | WHY IT IS HERE                                                       |
|-------|--------------------------------------------------------------------------------------------|----------------------------------------------------------------------|
| 1     | New tasks arrive.                                                                          | lifelong workload                                                    |
| 2     | Task states advance; completed tasks are recorded.                                         | a robot at its pickup becomes loaded                                 |
| 3     | Free robots are matched to pending tasks (§9).                                             | assignment decides <i>where</i> traffic will go                      |
| 4     | Waypoint, route distance $r_i$ , proximity mode, and the weights $\beta_i, \gamma_i$ (§4). | how strongly this robot should care about direction <i>right now</i> |
| 5     | Routes are recomputed; each robot's preferred compass direction is set, with hysteresis.   | the route is what carries direction into the one-step score          |
| 6     | Aisle entrance queues and per-robot direction requests.                                    | robots <i>request</i>                                                |
| 7     | Every aisle re-decides its direction (§6).                                                 | aisles <i>decide</i>                                                 |
| 8     | Entry reservations are expired and granted (§7).                                           | capacity admission                                                   |
| 9     | Deadlock detection and progressive recovery (§10).                                         | uses last step's wait-for graph                                      |
| 10    | Priorities $p_i(t)$ computed; robots sorted (§3).                                          | who gets to choose first                                             |
| 11–12 | PIBT: each robot in priority order picks its move, pushing others as needed (§8).          | the only step that decides collisions                                |
| 13    | The joint plan is validated: no two robots in one cell, no swaps.                          | a safety assertion, on by default                                    |
| 14–16 | Everyone moves at once; counters and metrics are logged.                                   |                                                                      |

Note what is *not* in step 12: nothing about direction. By the time PIBT runs, the aisle layer has already spoken, and it spoke by changing numbers in the score, not by removing options.

### 3 Who chooses first: priority

PIBT is a priority scheme, so the priority function is the policy for whose journey matters. It is a sum of a class term and four continuous terms.

$$p_i(t) = P_{\text{class}}(i) + P_{\text{aisle}} \cdot [i \text{ is inside an aisle}] + k_u \cdot U_i(t) + k_w \cdot W_i + k_b \cdot B_i + \varepsilon_i$$

$U_i$  task urgency ·  $W_i$  steps waited ·  $B_i$  steps blocked ·  $\varepsilon_i$  a fixed per-robot tie-breaker, so the order is deterministic.

| CLASS           | P <sub>CLASS</sub> | MEANING                                      |
|-----------------|--------------------|----------------------------------------------|
| emergency       | 400                | robot is in deadlock recovery                |
| loaded          | 300                | carrying, heading to delivery                |
| pickup          | 200                | heading to a pickup                          |
| repositioning   | 100                | parked / moving out of the way               |
| free            | 0                  | idle, no task                                |
| inside an aisle | +50                | added on top: get the corridor cleared first |

### THE INTUITION

Classes are 100 apart and waiting is worth  $k_w = 5$  per step, so a robot cannot be starved by class alone: after  $(400 - 0)/5 =$

### 80 STEPS OF WAITING AN IDLE ROBOT OUTFRANKS ANYTHING

. That ratio is the *fairness horizon*, and it is the only thing standing between "loaded robots first" and permanent starvation of everyone else. Being blocked counts double ( $k_b = 10$ ) because a blocked robot is usually the head of a queue: promoting it unjams the robots behind it too.

## 4 How much a robot should care about direction

A robot 30 cells from its waypoint should happily follow the traffic scheme. A robot one cell away should ignore it and arrive. So the direction weights are functions of the remaining route distance  $r_i$ , not constants.

mode = TRANSIT if  $r_i > R_{\text{far}}$ , APPROACH if  $r_i > R_{\text{near}}$ , else ARRIVAL

$\beta_i = \beta_{\text{max}} \cdot \min(1, \max(0, r_i - R_{\text{near}}) / \max(1, R_{\text{far}} - R_{\text{near}}))$

$\gamma_i = \gamma_{\text{strong}}$  (TRANSIT),  $\frac{1}{2}(\gamma_{\text{strong}} + \gamma_{\text{weak}})$  (APPROACH),  $\gamma_{\text{weak}}$  (ARRIVAL)

Defaults  $R_{\text{near}} = 2$ ,  $R_{\text{far}} = 8$ ,  $\beta_{\text{max}} = 3$ ,  $\gamma_{\text{strong}} = 2$ ,  $\gamma_{\text{weak}} = 0.5$ .

$\beta_i$  ramps linearly from 0 at two cells out to its ceiling at eight and stays there.  $\gamma_i$  is the pull to stay in the aisle you are already in — useful while transiting, harmful when your shelf is one cell to the side, so it decays too. Both are computed once per robot per timestep (step 4) and then held fixed while every candidate move is scored.

The robot's **preferred compass direction** is simply the orientation of the first edge of its route. Hysteresis keeps that preference from flapping: it is only re-committed at an intersection, when the robot is not in TRANSIT, or when it has been blocked for  $t_{\text{blocked}} = 10$  steps.

## 5 The score: one number per candidate move

This is the heart of the planner. For robot  $i$  considering moving to cell  $v$ :

$S_i(v) = \alpha \cdot \Delta_i(v) + \beta_i \cdot [v \text{ is the preferred direction}] + \gamma_i \cdot [v \text{ is in the same aisle}] - \lambda_{\text{turn}} \cdot P^{\text{turn}} - \mu \cdot C_i(v) - v \cdot P^{\text{wait}} - \xi \cdot [v \text{ is a bottleneck}] - \zeta_{\text{flow}} \cdot [\text{counterflow}] - \zeta_{\text{ticket}} \cdot [\text{no reservation}]$

| TERM                        | DEFAULT   | WHAT IT IS, AND WHY                                                                                                                                                                                                             |
|-----------------------------|-----------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| $\alpha \cdot \Delta_i$     | 10        | <b>Progress.</b> $\Delta$ = route distance from here – route distance from $v$ , i.e. <b>+1, 0 or –1</b> . This dominates everything, which is the point: the planner is a shortest-path follower that has been taught manners. |
| $\beta_i$                   | 0 ... 3   | Move agrees with the robot's own preferred heading. Decays to 0 near the waypoint (§4).                                                                                                                                         |
| $\gamma_i$                  | 0.5 ... 2 | Stay in the aisle you are in. Zeroed when the robot is standing on an intersection — an intersection is where you are <i>meant</i> to change aisle.                                                                             |
| $\lambda_{\text{turn}}$     | 0.5       | Turn cost: 0 straight, 1 for a turn, $\eta = 2$ for a reversal. Real robots pay for rotation; this stops zig-zag paths.                                                                                                         |
| $\mu \cdot C_i$             | 1         | Congestion, in $[0, 1]$ (§5.1). Steers around jams before entering them.                                                                                                                                                        |
| $\nu \cdot p^{\text{wait}}$ | 0.2       | Waiting in place costs 1 — <i>unless</i> the robot is already at its waypoint, where waiting is free.                                                                                                                           |
| $\xi$                       | 1         | Bottleneck cell (cut vertex, bridge, dead end). Don't loiter in the doorway.                                                                                                                                                    |
| $\zeta_{\text{flow}}$       | 8         | <b>Counterflow.</b> The move opposes the aisle's committed direction.                                                                                                                                                           |
| $\zeta_{\text{ticket}}$     | 8         | <b>No ticket.</b> Entering a directional aisle without a valid reservation, or one that is already full.                                                                                                                        |

#### WHY Z = 8 AND NOT 8000

Read the numbers as a sentence. A step of progress is worth 10. Every polite term put together — heading, aisle continuity, no turn, empty, not a bottleneck — is worth at most  $3 + 2 = 5$ . So:

- **A  
LEGAL**  
move that makes progress scores  $10 +$  up to 5. It always wins.
- **A  
COUNTERFLOW**  
move that makes progress scores  $10 - 8 =$   
+2  
.
- **WAITING**  
scores  $-0.2$  (and worse if the cell is congested).

So a robot with a legal way forward never goes counterflow, and a robot with no legal way forward goes counterflow rather than sit in a queue. That is the whole policy, and it falls straight out of  $\alpha > \zeta > \beta > \gamma > \mu \approx \xi > \lambda > \nu$ .

## 5.1 Congestion

$$C_i(v) = (\omega_l \cdot C_{\text{local}} + \omega_a \cdot C_{\text{aisle}} + \omega_d \cdot C_{\text{down}}) / (\omega_l + \omega_a + \omega_d)$$

$C_{\text{local}}$  = robots within Manhattan radius 3 of  $v$ , divided by the passable cells in that diamond ·  $C_{\text{aisle}}$  = occupancy / capacity of  $v$ 's aisle ·  $C_{\text{down}}$  = fraction of the next 5 route cells beyond  $v$  that are occupied. All three weights default to 1.

All three components are **ratios in [0, 1]** and the weights are normalised, so  $C$  itself is in  $[0, 1]$  and  $\mu \cdot C$  can never reach the scale of  $\alpha \cdot \Delta$ . This is not cosmetic: if  $C_{\text{local}}$  were a raw count (up to ~13 robots in that diamond) the congestion penalty alone would outweigh a step of progress and the robot would prefer to stand still in empty space.

---

## 6 The aisle layer: direction as a traffic signal

The governing rule: **robots generate directional requests, aisles make directional decisions**. No robot ever sets an aisle's direction; it only votes.

### 6.1 The vote

Each robot's route names the aisles it needs and the direction it needs them in (index increasing = FORWARD). Its vote has a weight:

$$d_{i,a} = w_u \cdot U_i + w_w \cdot (\max(W_i, \text{stalled steps}) + B_i) + w_p / (1 + \text{dist}(x_i, \text{nearest end of } a)) - w_l \cdot (\text{route length}) - w_c \cdot (\text{load of } a)$$

Defaults  $w_u = 1$ ,  $w_w = 0.5$ ,  $w_p = 2$ ,  $w_l = 0.05$ ,  $w_c = 0.5$ .  $S_a^+$  and  $S_a^-$  are the sums of  $d_{i,a}$  over the robots asking for FORWARD and REVERSE.

Three details carry weight. **Proximity** is  $1/(1+d)$ , so a robot at the mouth of the aisle counts for 2 and one ten cells away counts for 0.18 — imminent demand decides. **Stalled route progress counts as waiting**, not just standing still: a robot shuffling in place while getting no closer is starving, and this is what eventually flips an aisle against it. And demand is **signed against congestion**, so an aisle already full does not attract even more traffic.

### 6.2 The decision

Let  $I = S_a^+ - S_a^-$  and let  $\theta = 5$  be the dead band. Every timestep, for each manageable aisle:

```
# state machine: OPEN -> FORWARD/REVERSE -> DRAINING -> ...
if state is DRAINING:
    if occupancy > 0 and (t - draining_since) <= max_drain_time:    # 30
        hold                                                         # nobody new may enter
    elif drained: commit(pending_direction)                          # the flip we drained for
    else:          reopen as OPEN                                     # drain timed out; do not block forever

elif occupancy > 0 and direction is committed:
    hold the direction, but
    if opposing_imbalance > 0: begin drain
    if (t - direction_since) >= max_lock and opposing > 0: begin drain # maximum green

else:
    # empty, or no direction yet
    if t < lock_until: hold                                           # minimum green: 20 steps
    elif I > 0: commit FORWARD
    elif I < -0: commit REVERSE
    else: OPEN (no direction at all)
```

## TWO CLOCKS, AND WHY BOTH ARE NEEDED

### MINIMUM GREEN

( `minimum_aisle_lock_time` = 20, and never more than twice the aisle's length) plus the dead band  $\theta$  is hysteresis: it stops an aisle flipping every time the demand wobbles.

### MAXIMUM GREEN

( `maximum_aisle_lock_time` = 40) is the other half, and it is easy to leave out. Hysteresis bounds how *soon* a direction may change; nothing bounds how *long* it may persist. Pickups on one side of the map and deliveries on the other produce near-balanced demand — the normal case — so the imbalance never leaves the dead band and one direction is held forever while the other queue starves. Past 40 steps, *any* opposing demand forces a drain and a flip. Every such flip is counted as a `starvation_flip`, so the mechanism is visible in the metrics rather than a silent internal.

**Drain before reverse** is the safety rule: an aisle never reverses while robots are still inside it, because that would turn every one of them into a counterflow violator at once. It enters DRAINING, refuses new entrants, waits for the corridor to empty, and only then commits the pending direction. If it cannot empty within 30 steps — someone inside is itself stuck — the aisle reopens rather than blocking traffic indefinitely.

### 6.3 What "counterflow" actually means

`violates_aisle_direction` is a *predicate*, not a veto. Its answer becomes  $\zeta_{\text{flow}}$  in the score. It is true when the aisle has a committed direction and:

- the robot is **outside** and would enter against the direction (or the aisle is DRAINING — nobody enters a draining aisle); or
- the robot is **inside** and moves against the direction *without* heading for the nearer end.

That last exemption is what lets DRAINING terminate: a robot caught facing the wrong way is always allowed to back out toward the near exit for free.

---

## 7 Admission: reservations and capacity

Direction says *which way*; capacity says *how many*. They are separate mechanisms and separate flags — over-filling a single-file corridor jams it regardless of which way it points.

- Capacity.** Default model "drain": an aisle of length  $L$  holds  $\min([L], \text{max\_drain\_time} - L)$  robots, capped at 10. The reasoning is the exit, not the volume: the last robot in still has  $L$  cells to cross and every robot ahead adds a step, so a queue that cannot clear within the drain window is a queue that will jam.
- Tickets.** A robot within 3 cells of an entrance, whose requested direction matches, and for which `occupancy + outstanding_tickets < capacity`, is granted a `Reservation` with a TTL of 15 steps. Tickets are handed out in priority order, expire on their own, and are dropped as soon as the robot is inside or has re-routed.
- Open aisles need no ticket.** Requiring one to cross an OPEN aisle would throttle every crossing on a map of short corridors. The occupancy cap still applies there.

Entering without a valid ticket costs  $\zeta_{\text{ticket}} = 8$  — again a price, not a wall.

---

## 8 PIBT: the only layer that decides collisions

Everything above changed the *order* of a robot's candidate moves. Nothing above removed one. PIBT now takes that ordered list and resolves the joint plan.

The candidate set is  $C_i = \text{neighbours of } x_i \cup \{x_i\}$ . A candidate is dropped only for reasons of physics or safety:

| REJECTION       | TEST                                                                       |
|-----------------|----------------------------------------------------------------------------|
| off-graph       | the cell is a wall or outside the map                                      |
| vertex conflict | the cell is already reserved this timestep, or its occupant is staying put |
| swap conflict   | the move would exchange cells with the robot that pushed you               |

```
PIBT(i, parent):
    for v in candidates(i) sorted by descending S_i(v):      # $5
        if v is reserved: continue
        reserve v for i
        j = occupant(v)
        if j is none or j has already planned elsewhere: return VALID
        # priority inheritance: j must move, and it must move now
        if PIBT(j, parent=i) is VALID: return VALID
        # backtracking: undo and try i's next-best candidate
        un-reserve v
    # nothing worked: stay put, and record the block
    i.next = i.position; return INVALID
```

### WHY THE AISLE LAYER MUST NOT DELETE CANDIDATES

PIBT's progress argument is that the highest-priority robot always gets its best move, because any robot in the way can be *pushed* into some adjacent cell. That argument consumes free neighbours. A one-way rule removes, on average, half the neighbours of every cell in a corridor — precisely the escape moves the recursion needs. The result is not "slightly more careful traffic": it is a planner whose inheritance chains dead-end and whose corridors fill with robots that are individually obeying the rules. Measured, that is a **1.9× TO 3.7× LOSS OF THROUGHPUT** versus pricing the same rule ([matrix-comparison.pdf](#)). The soft version keeps every candidate and merely reorders them, so the progress argument survives intact.

`hard_direction_constraints` restores the deleting behaviour. It exists only so that comparison can be run.

---

## 9 Task assignment

Greedy minimum-cost matching between free robots and released tasks, re-run every timestep. It matters because assignment decides where traffic *will* be, minutes before the movement layer has to cope with it.

```

$$J(i, \tau) = a \cdot d(x_i, \text{pickup}) + b \cdot d(\text{pickup}, \text{delivery}) + g \cdot C_{\text{route}} + \delta \cdot \min(W_{\tau}, 60) + e \cdot T_{\text{direction}} + z \cdot B$$

$$a = 1, b = 0.5, g = 12, \delta = -0.5 \text{ (negative: older tasks are preferred)}, e = 1, z = 1.$$

```

$T_{\text{direction}}$  is the expected wait for aisles currently pointing the wrong way (about half a lock time plus the occupancy of each such aisle) — the assignment layer *knows* about the traffic signals and prices the detour honestly.  $B$  is the fraction

of the route made of bottleneck cells. Two caps matter: the task-age term is clipped at 60, or in a lifelong run it grows without bound and the match degenerates to oldest-task-first; and  $C_{\text{route}}$  is a per-cell mean, so a long empty route never looks worse than a short jammed one.

## 10 When it jams anyway

Three stall signals, and a deliberate rule about combining them: no route progress for `t_blocked` = 10 steps; a repeated local configuration (the robot's last 4 positions equal the 4 before them); and a cycle in the wait-for graph, where  $i \rightarrow j$  means " $i$  asked  $j$  to move and  $j$  could not".

**No progress alone is not a deadlock.** In dense lifelong traffic a robot queues for ten steps as a matter of course. So it is only a precondition: a group must also show a cycle or a repeated configuration (`require_deadlock_corroboration`). Without that check, recovery fires on healthy traffic and levels 5–7 below destroy more throughput than they save. Groups are then formed either from a wait-for cycle or from spatially clustered blocked robots.

Recovery is **progressive**: a group that is still stuck escalates one level per timestep, so cheap remedies get their chance first.

| LV | REMEDY                                                            | COST                              |
|----|-------------------------------------------------------------------|-----------------------------------|
| 1  | recompute the group's routes                                      | free                              |
| 2  | re-decide the directions of the aisles involved                   | local                             |
| 3  | release stale reservations                                        | local                             |
| 4  | raise the blocked robots to emergency priority (400)              | reorders traffic                  |
| 5  | allow a temporary reverse move (ignore direction for a few steps) | breaks the traffic scheme locally |
| 6  | send robots to escape vertices (passing bays)                     | abandons the route                |
| 7  | run a local fallback planner                                      | expensive                         |

A group that survives level 7 is counted as `unrecovered` and the strongest remedy keeps being applied. That counter is reported rather than hidden.



---

## 11 The knobs that actually change behaviour

| PARAMETER                                   | DEFAULT | TURN IT UP / OFF AND...                                                                                          |
|---------------------------------------------|---------|------------------------------------------------------------------------------------------------------------------|
| <code>direction_control</code>              | aisle   | "robot" : robots prefer a heading but no aisle ever commits. "none" : plain PIBT with congestion and turn costs. |
| <code>zeta_counterflow</code>               | 8       | > 10 makes direction effectively hard (progress can no longer buy it). 0 removes the traffic scheme's teeth.     |
| <code>hard_direction_constraints</code>     | off     | On: direction becomes a rejection again. This is the ablation the method is named against.                       |
| <code>maximum_aisle_lock_time</code>        | 40      | Higher: fewer flips, more starvation. Unbounded: aisles lock forever under balanced demand.                      |
| <code>direction_switch_threshold</code>     | 5       | The dead band. Lower: twitchy signals. Higher: sluggish response.                                                |
| <code>reservations</code>                   | on      | Capacity admission. Independent of direction control.                                                            |
| <code>require_deadlock_corroboration</code> | on      | Off: recovery fires on ordinary queueing and costs throughput.                                                   |
| <code>directional_aisle_min_length</code>   | 4       | Short links are never made one-way; making them one-way fragments the network.                                   |
| <code>direction_aware_routing</code>        | off     | On: routes are A* with a penalty of 6 per edge against the flow, so robots detour instead of queueing.           |

---

## 12 Formula → code

| WHAT                                 | WHERE                                                                    |
|--------------------------------------|--------------------------------------------------------------------------|
| priority $p_i(t)$                    | <code>priority.py:compute_priority</code>                                |
| $\beta_i, \gamma_i$ , proximity mode | <code>scoring.py:compute_direction_weight, compute_aisle_weight</code>   |
| score $S_i(v)$                       | <code>scoring.py:CandidateScorer.score</code>                            |
| $\zeta$ terms                        | <code>scoring.py:CandidateScorer.aisle_penalty</code>                    |
| congestion $C_i(v)$                  | <code>congestion.py:CongestionModel.congestion</code>                    |
| demand $S_a^\pm$                     | <code>aisle_manager.py:compute_directional_demand</code>                 |
| signal state machine                 | <code>aisle_manager.py:update_aisle_direction</code>                     |
| tickets and capacity                 | <code>aisle_manager.py:update_aisle_reservations, can_enter_aisle</code> |
| counterflow predicate                | <code>aisle_manager.py:violates_aisle_direction</code>                   |
| PIBT recursion                       | <code>pibt.py:PIBTPlanner.pibt</code>                                    |
| hard-rejection rules                 | <code>pibt.py:feasible_candidates</code>                                 |
| assignment cost $J$                  | <code>assignment.py:assignment_cost</code>                               |
| detection and the ladder             | <code>deadlock.py:detect_deadlocked_groups, recover_from_deadlock</code> |
| the timestep                         | <code>simulator.py:Simulator.step</code>                                 |
| every default quoted here            | <code>config.py:Params</code>                                            |

Ask the running planner why a robot did what it did: `pibt.py:explain_candidates` returns, per candidate, the rules that rejected it, the penalties it merely carried, and its score. A robot that is stalled almost always has one repeated reason.